



HELLENIC REPUBLIC
National and Kapodistrian
University of Athens
— EST. 1837 —



WRIGHT STATE
UNIVERSITY



Aiteam
NATIONAL & KAPODISTRIAN
UNIVERSITY OF ATHENS

Methodologies for Geospatial Knowledge Graph Construction

Sergios-Anestis Kefalidis
Kostas Plas
Manolis Koubarakis
Cogan Shimizu

Tutorial Outline

- Introduction to knowledge graph construction
- Construction of Geospatial Knowledge Graphs
- Interlinking in Geospatial Knowledge Graphs
- Geospatial Interlinking
- Using the Topos framework to construct GeoKGs

Introduction to Knowledge Graph Construction

- In the previous part of the tutorial we showcased **geospatial knowledge graphs** and how useful they can be in a wide array of domains and applications
- Some questions might remain!
 - How do we construct a GeoKG?
 - Can we imbue GeoKGs with thematic knowledge?
 - What ways are there to optimize GeoKGs for better responsiveness

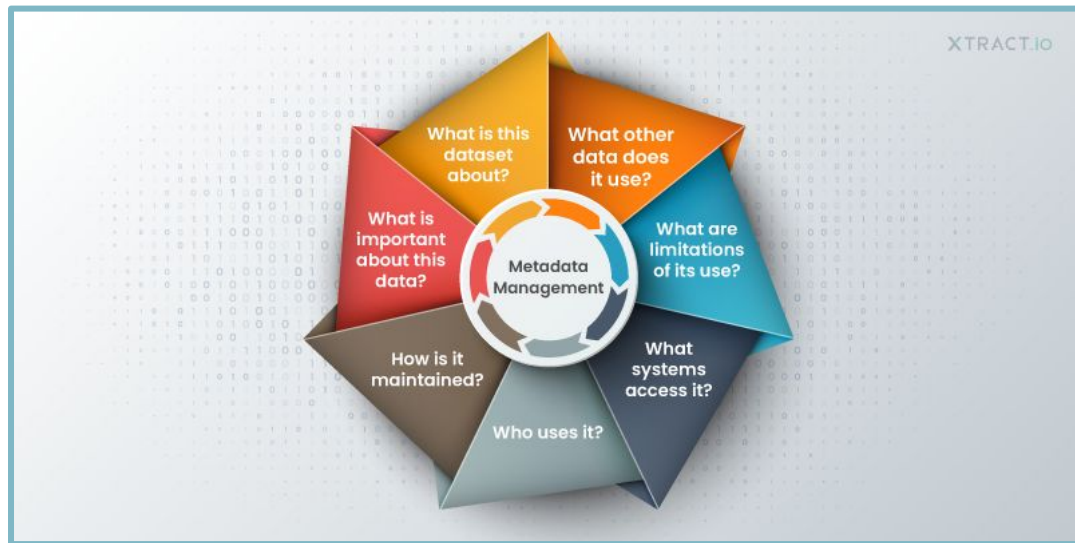
Introduction to Knowledge Graph Construction

- When constructing a knowledge graph we need to keep in mind:
 - Metadata Management
 - Data Preprocessing
 - Ontology Management
 - Knowledge Extraction
 - Entity Resolution
 - Completion
 - Quality Assurance

Construction of Knowledge Graphs: Current State and Challenges Marvin Hofer, Daniel Obraczka, Alieh Saeedi, Hanna Köpcke and Erhard Rahm

Introduction to Knowledge Graph Construction: Metadata Management

- Before constructing and utilizing our knowledge graph, it is important to maintain **metadata** for our data sources
- We can maintain and look-up **metadata repositories** to maintain our **metadata**
- Especially when it comes to issues such as versioning, which is very important to **geospatial data**, which are dynamic and may change very frequently (E.g. Geofabrik spatial repository is updated daily from OSM)



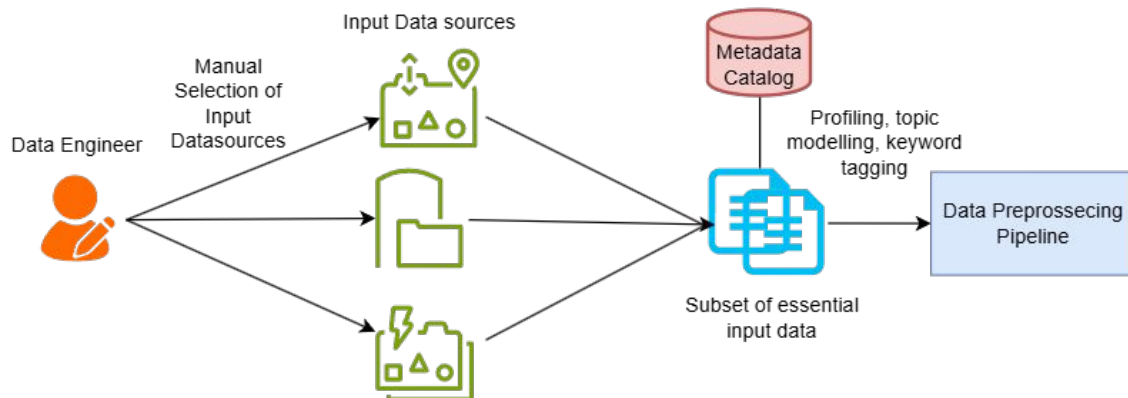
Introduction to Knowledge Graph Construction: Data Preprocessing (1)

- One of the most critical issues in knowledge graph construction! 
- Before starting the construction of our KG, we must select **what kind of data** we will be using.
- Remember that when selecting our data sources, we don't need to integrate them in their entirety (e.g. for a pandemic specific KG, only health-related parts of existing KGs, such as DBpedia, may be needed.)
- Important issues to keep in mind:
 - **Quality** of our sources
 - **Trustworthiness** of our sources
- This is generally a **manual process** (but metadata catalogs may help automate it)

Introduction to Knowledge Graph Construction: Data Preprocessing (2)

● Data Acquisition

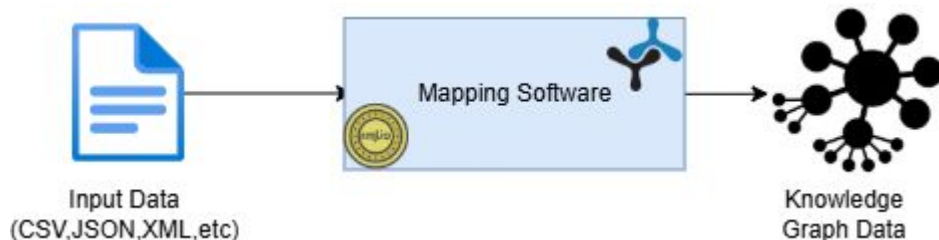
- The input data sources might be available in different formats (CSV, XML, JSON, RDF etc.)
- What about spatial data? (Put a pin on this 📌)
- When dealing with a variety of different formats, it is common practise to use **data source adapters**
- How do we deal with changing data:
 - Store different complete versions of the knowledge graph
 - Annotate triples with tags that show triple age and validity



Introduction to Knowledge Graph Construction: Data Preprocessing (3)

- **Transformation and Mapping**

- After collecting and parsing our input data we need to transform them into KG data!
- A lot of tools are available for this process, but for semi-structured data **RML** (RDF Mapping Language) **mappings**
- SPARQL-CONSTRUCT queries might also be used for RDF to RDF mappings.



- *RDF Mapping Language (RML)*, <https://rml.io/>

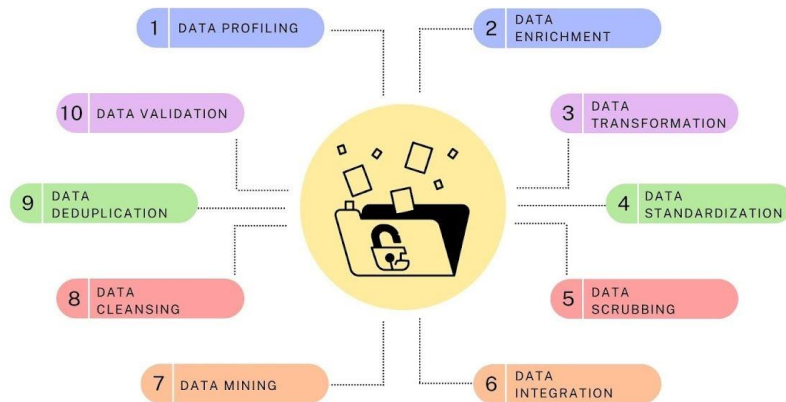
- *Apache Jena*, <https://jena.apache.org/>

Introduction to Knowledge Graph Construction: Data Preprocessing (4)

- **Data Cleaning**

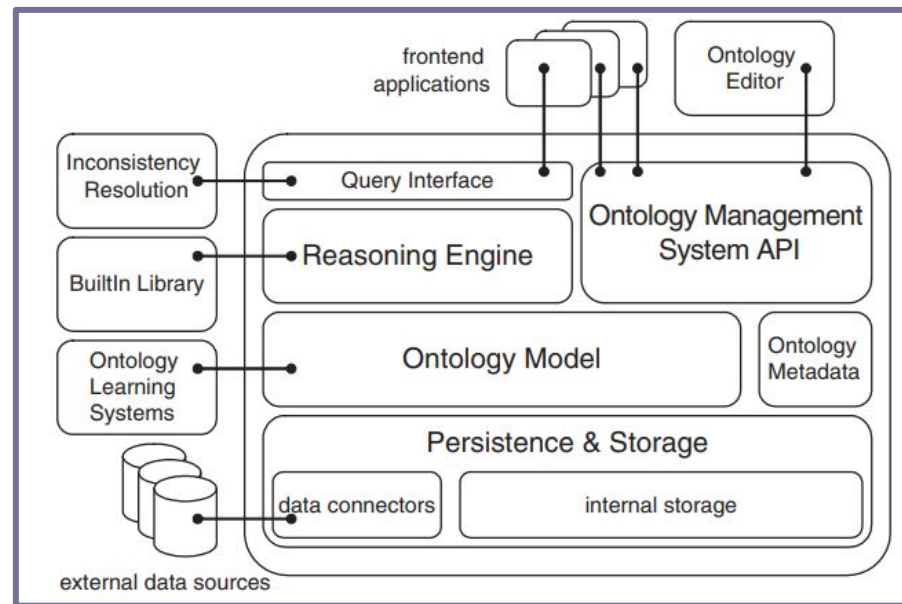
- Process of detecting and removing errors and inconsistencies from data
- Both **structural** and **semantic** inconsistencies must be accounted for
- Traditionally, rule based methods were used, but LLMs are quite helpful in this task

DATA CLEANING TECHNIQUES



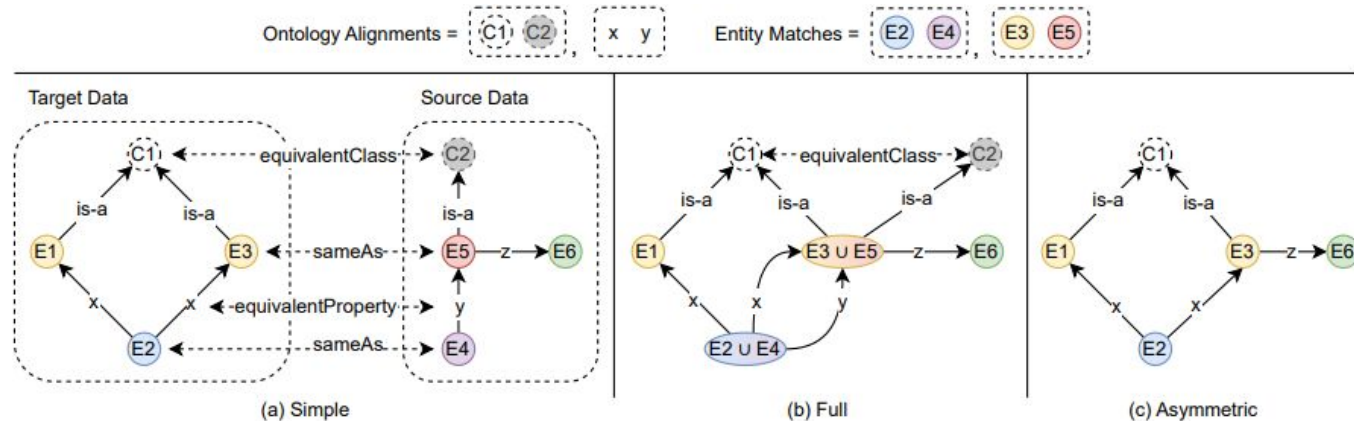
Introduction to Knowledge Graph Construction: Ontology Management (1)

- The incremental process of creating, extending and maintaining an **ontology**
- Mainly a **manual** task, but there are efforts to provide semi-automated **ontology management pipelines**
- It's good practice to build on top of existing ontologies like Wikidata or DBpedia, or domain specific ontologies like Bio2RDF.
- **Ontology Learning:**
 - Unstructured text data to schema (LLM application)
 - Structured data like relational databases (Rules and Mappings)



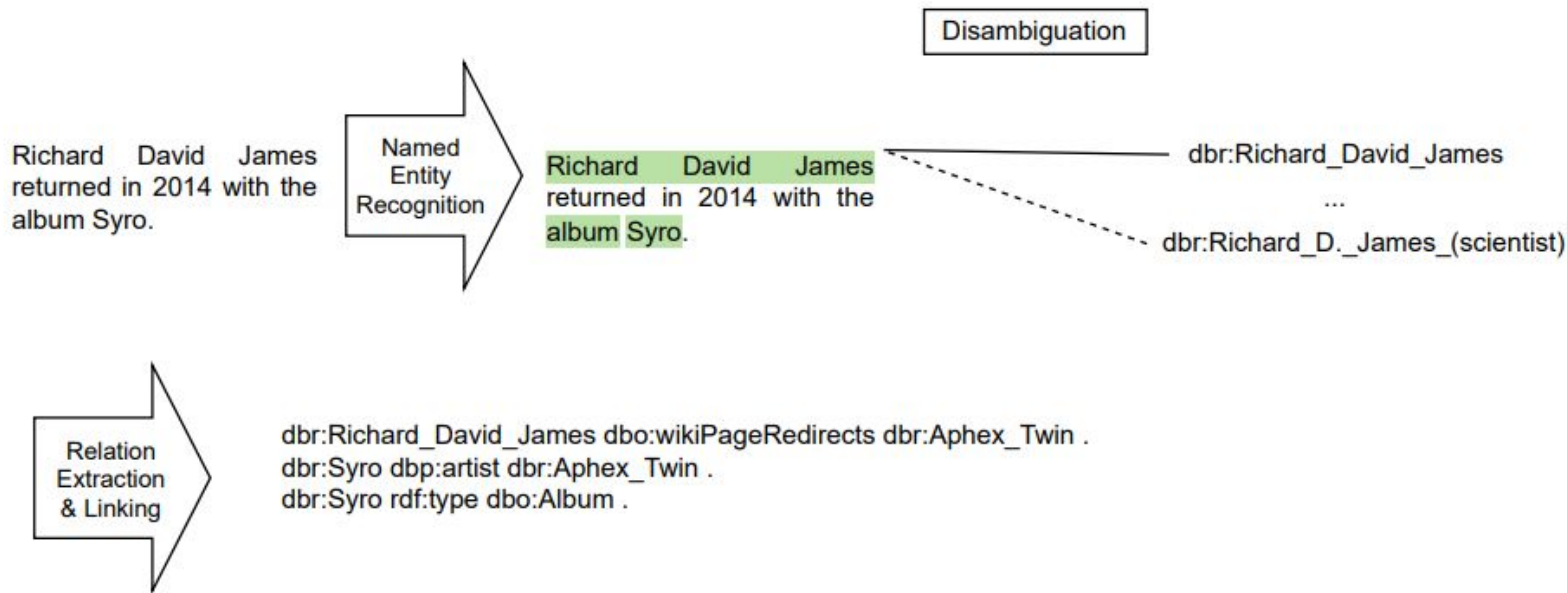
Introduction to Knowledge Graph Construction: Ontology Management (2)

- **Ontology/Schema matching and integration**
 - Consolidating information from different data sources requires harmonizing the ontologies of the sources
 - Techniques like semantic similarity or LLM-based content extraction are used for matching
 - Ontology integration uses match alignments to merge ontologies through various types of merges



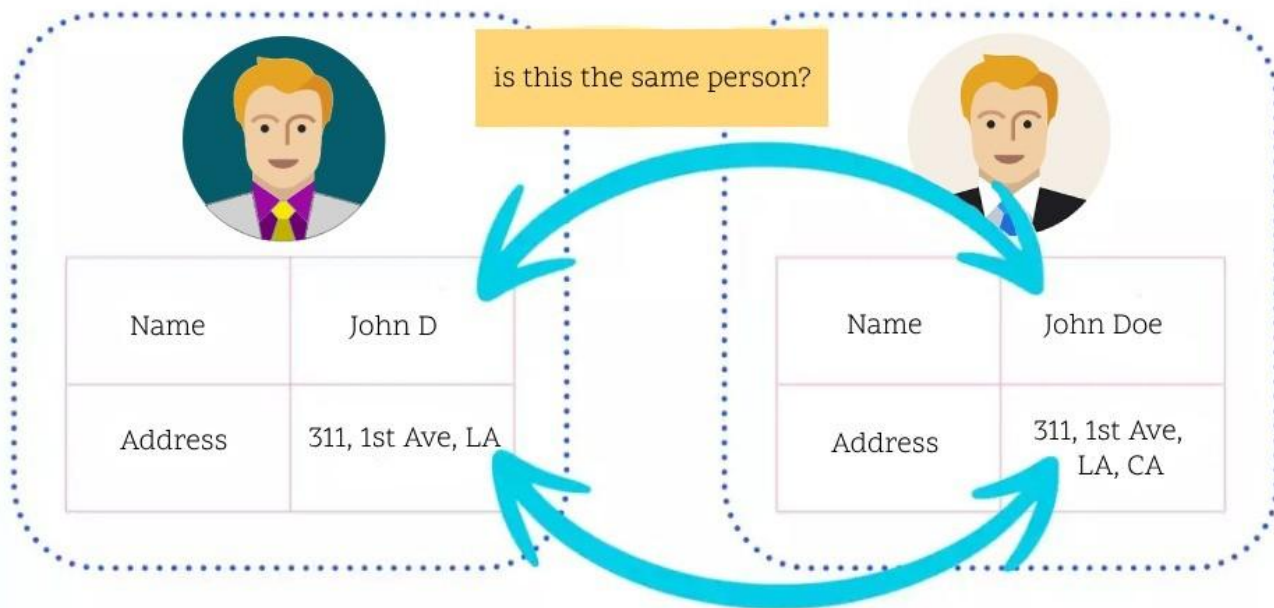
Introduction to Knowledge Graph Construction: Knowledge Extraction

- Process of obtaining structured info from unstructured textual data or semi-structured data.
- Methods like NER, Disambiguation and Relation Extraction & Linking



Introduction to Knowledge Graph Construction: Entity Resolution (1)

Entity Resolution: What? How? Why?

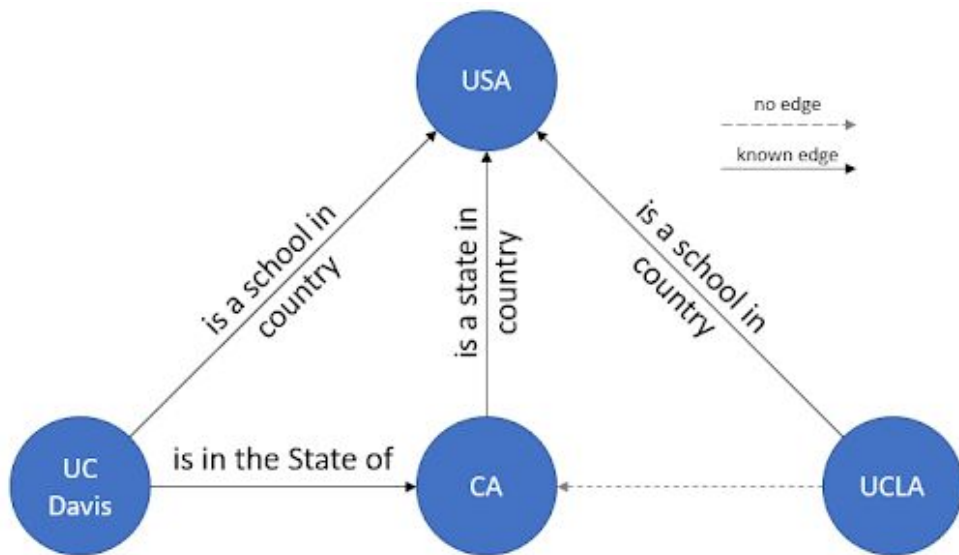


Introduction to Knowledge Graph Construction: Entity Resolution (2)

- **Entity Resolution** (ER) identifies duplicate records across datasets using a blocking–matching–clustering pipeline; modern methods use deep learning, KG embeddings, and LLMs (e.g., AutoBlock, DeepBlocker, Sudowoodo) for higher accuracy.
- **Incremental ER** matches new or streaming entities to existing KG entities, using clusters for better performance and repair, with parallel and schema-agnostic techniques improving scalability.
- **Advanced ER** applies **transformers, contrastive learning, and multimodal models** (e.g., visual-text fusion, multilingual BERT) for complex and geospatial entity matching.
- **Entity Fusion** merges duplicates into a single clean record, resolving attribute conflicts via ignorance, avoidance, or resolution strategies using trust, recency, or statistical heuristics.

Introduction to Knowledge Graph Construction: Completion

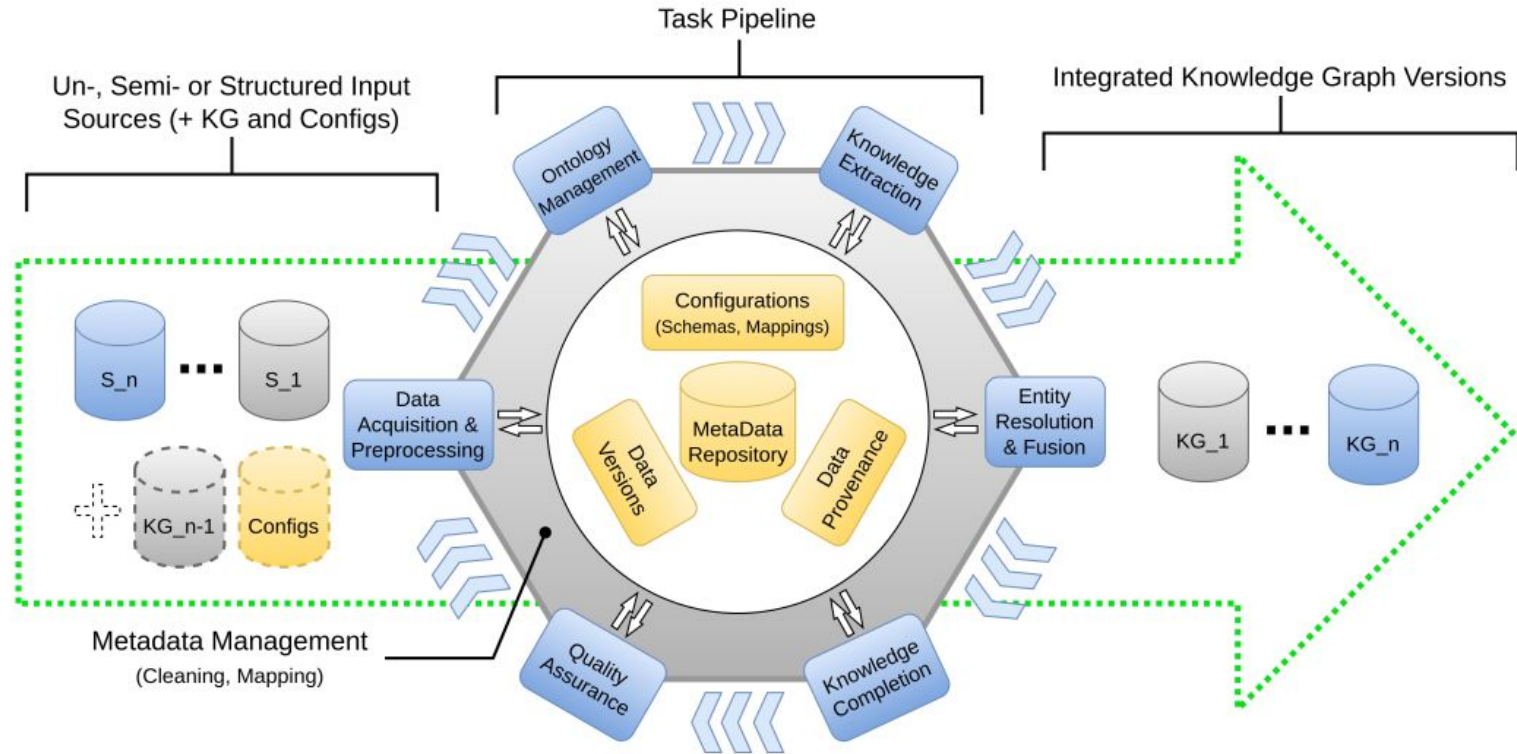
- Involves adding **new entries** to the graph using existing relationships
- **Type completion**: Annotating nodes with types (class declarations or literal types)
- **Link prediction**: Discover and integrate missing links and relationships between entities in the knowledge graph



Introduction to Knowledge Graph Construction: Quality Assurance

- **KG quality assurance** maintains credibility and usability through continuous evaluation, repair, and refinement.
- **Quality** is managed across the KG lifecycle. Starting from source selection to fusion, by utilizing metadata and provenance to **ensure validity**.
- **Quality needs vary by use case**: For example accuracy is important for finance/e-commerce, but approximation is preferable for recommendations.

A Complete Knowledge Graph Construction Pipeline



Construction of Knowledge Graphs: Current State and Challenges Marvin Hofer, Daniel Obraczka, Alieh Saeedi, Hanna Köpcke and Erhard Rahm

Introduction to Knowledge Graph Construction

- Bellow are some tools useful for knowledge graph construction

Tool	Description & use case
<u>Protégé</u>	An ontology-editor and modelling environment (open-source) to define the schema/ontology (classes, relations, constraints) for a KG
<u>Morph-KGC</u>	A materialisation engine for converting heterogeneous data (CSV, JSON, XML, relational, etc) into RDF/triple form using mapping languages (R2RML / RML).
<u>Sangrahaika</u>	A tool for annotation and querying of KGs: helps annotate entities & relations from text corpora and build a KG.
<u>KG-Hub</u>	A platform / framework (especially in the biological domain) for building, versioning, publishing and reusing KGs, with ETL pipelines + ontologies + cloud artifact storage.
<u>NeOn Toolkit</u>	An older ontology-engineering environment (plug-in based) useful for modelling ontologies – often part of KG workflows for the TBox part.

Construction of Geospatial Knowledge Graphs (1)

- When constructing a **geospatial** knowledge graph, the first thing that must come to mind is **geospatial data**!
- Some questions to keep in mind
 - What is the **scope** of our geospatial knowledge graph?
 - What **data sources** do we use?
 - What are the available **data formats**?

Construction of Geospatial Knowledge Graphs (2)

- Which steps change specifically for GeoKG:
 - Metadata Management
 - Data Preprocessing
 - Ontology Management
 - Knowledge Extraction
 - Entity Resolution and Interlinking
 - Completion
 - Quality Assurance

Construction of Geospatial Knowledge Graphs (3)

- As with any KG, one of the initial steps of creating a GeoKG is choosing our data sources.
- When working with spatial data, what kind of data sources are available?



GeoNames



Construction of Geospatial Knowledge Graphs (4)

- One of the initial steps of creating a GeoKG is choosing our data sources.
- When selecting spatial data sources we have to decide what kind of spatial information we need to use:
 - Points-of-Interest, (Country or city specific data, [OSM](#), [DBpedia](#) etc)
 - Satellite image polygons ([NASA](#), [USGS](#), [Copernicus](#), etc.)
 - Administrative boundary data ([GADM](#), [GAUL](#))
 - Natural data like rivers or forests (Data from national services, OSM, [Global Forest Watch](#))
- And many, many more data sources

Construction of Geospatial Knowledge Graphs (5)

- After deciding what data is useful for our application (**Data Acquisition**), we need to transform them into RDF data.
- But in what formats are **spatial data** available in?
- Examples of the most common used spatial data formats:
 - GeoJSON
 - Shapefile
 - KML/KMZ
 - GML
 - Raster Data

Construction of Geospatial Knowledge Graphs: GeoJSON

- **GeoJSON** is a JSON-based format for encoding geographic data structures.
- It represents features like **points**, **lines**, and **polygons** with **coordinates**.
- Commonly used in web maps and GIS applications for easy data sharing and visualization.

```
{
  "type": "FeatureCollection",
  "features": [
    {
      "type": "Feature",
      "properties": {
        "name": "Nara",
        "country": "Japan"
      },
      "geometry": {
        "type": "Point",
        "coordinates": [-122.4194, 37.7749]
      }
    }
  ]
}
```


Construction of Geospatial Knowledge Graphs: Shapefile

- Shapefiles are a widely used vector data format created by ESRI to store geographic features such as points, lines, and polygons.
- They contain both geometry data (locations and shapes) and attribute data (descriptive information) for each feature.
- A shapefile is actually a set of related files — most importantly:
 - **.shp** (geometry),
 - **.shx** (index), and
 - **.dbf** (attribute table).

SHAPEFILES



Shape format —
stores geometry
(points, lines, polygons)



Shape index format
— stores positional
index of geometry



Attribute format —
tabular data in
dBase format



(Optional)
Projection format —
coordinate system info



(Optional)
Spatial index files



(Optional)
Character encoding
info for **.dbf** file

Construction of Geospatial Knowledge Graphs: KML/KMZ

- **KML** (Keyhole Markup Language) is an XML-based format used to represent geographic data for visualization in tools like Google Earth and Google Maps.
- It supports points, **lines**, **polygons**, and overlays along with metadata such as names, descriptions, and styles.
- **KMZ** is a compressed (zipped) version of a KML file, often used to bundle related resources like images or icons.

```
<?xml version="1.0" encoding="UTF-8"?>
<kml xmlns="http://www.opengis.net/kml/2.2">
  <Placemark>
    <name>San Francisco</name>
    <description>City in California, USA</description>
    <Point>
      <coordinates>-122.4194,37.7749,0</coordinates>
    </Point>
  </Placemark>
</kml>
```

Construction of Geospatial Knowledge Graphs: GML

- **GML** (Geography Markup Language) is another XML-based format developed by the **Open Geospatial Consortium** (OGC) for storing and exchanging geographic data.
- It represents geometries (points, lines, polygons, etc.) and their attributes in a structured, machine-readable way.
- GML is more complex and extensible than KML or GeoJSON, often used in GIS systems, data exchange, and spatial databases.

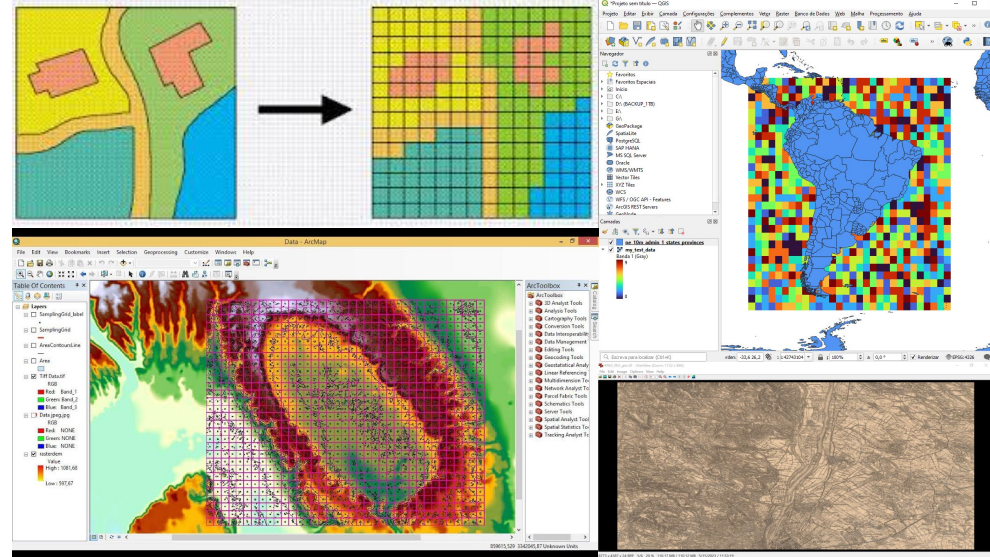
```
<?xml version="1.0" encoding="UTF-8"?>
<gml:FeatureCollection
  xmlns:gml="http://www.opengis.net/gml"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://www.opengis.net/gml
http://schemas.opengis.net/gml/3.1.1/base/gml.xsd">

  <gml:featureMember>
    <gml:City>
      <gml:name>San Francisco</gml:name>
      <gml:country>USA</gml:country>
      <gml:location>
        <gml:Point srsName="EPSG:4326">
          <gml:coordinates>-122.4194,37.7749</gml:coordinates>
        </gml:Point>
      </gml:location>
    </gml:City>
  </gml:featureMember>

</gml:FeatureCollection>
```

Construction of Geospatial Knowledge Graphs: Raster Data

- **Raster formats** store geographic data as a **grid of pixels** (cells), where each pixel represents a value such as elevation, temperature, or land cover.
- Each raster file contains spatial reference information (location, resolution, projection) that ties the pixel grid to real-world coordinates.
- Common raster formats include **GeoTIFF** (.tif), **JPEG2000** (.jp2), and **IMG** (.img) are often used in remote sensing and satellite imagery.



Construction of Geospatial Knowledge Graphs (6)

- After collecting our data sources and know their data formats, we can parse them into appropriate RDF formats, using techniques similar to the **Transformation and Mapping** techniques discussed in the previous section.
- RML mapping processor with geospatial capabilities (e.g Geotriples) can be utilized to transform the geospatial data sources in Knowledge Graph data files.
- Other useful tools for these kinds of transformations are GDAL and TripleGeo.
- A small problem still remains...
 - How do we represent **spatial information** in our GeoKGs

Construction of Geospatial Knowledge Graphs: Representation of Spatial Data (1)

- Spatial data in knowledge graphs is usually stored in **WKT** (Well-Known Text) format. **WKT** is a text-based format used to represent geometric shapes such as points, lines, and polygons.
- **Spatial triple stores**, that are used to store and query knowledge graphs, utilize **WKT** or its binary representation **WKB** (Well-Known Binary).
- **Note:** There are some systems that model spatial data in different formats like grid cells, which may be very useful for applications like urban planning.

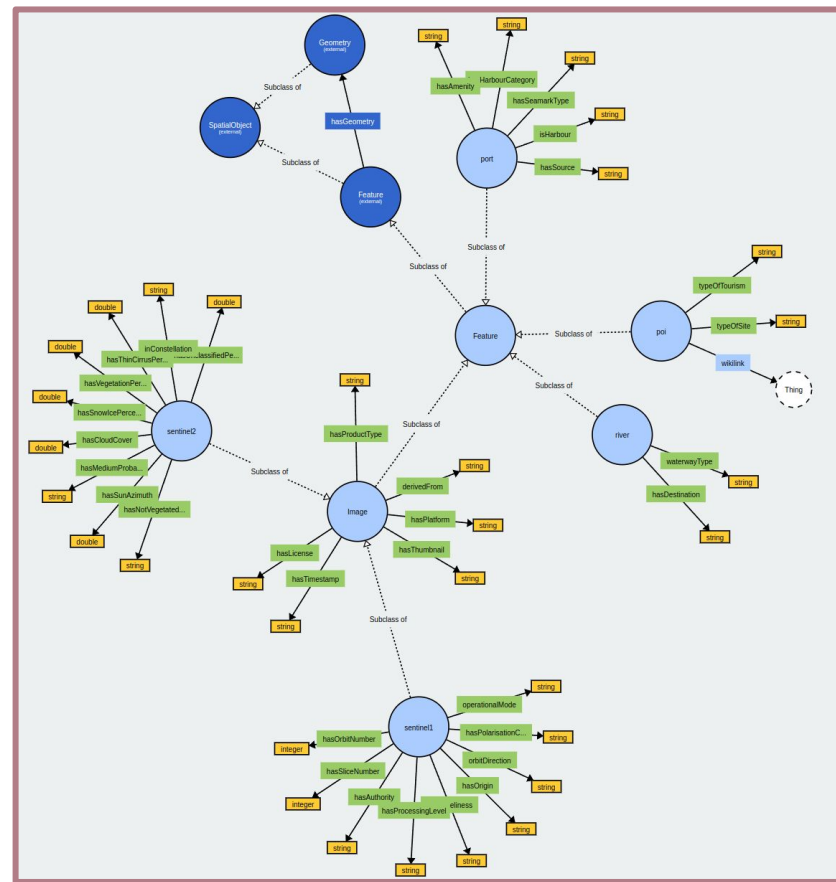
Construction of Geospatial Knowledge Graphs: Representation of Spatial Data (2)

Some examples of WKTs:

- Point: **POINT (30 10)**
- LineString: **LINESTRING (30 10, 10 30, 40 40)**
- Polygon: **POLYGON ((30 10, 40 40, 20 40, 10 20, 30 10))**
- MultiPoint: **MULTIPOINT ((10 40), (40 30), (20 20), (30 10))**
- MultiLineString: **MULTILINESTRING ((10 10, 20 20), (15 15, 30 15))**
- MultiPolygon: **MULTIPOLYGON (((30 20, 45 40, 10 40, 30 20)))**

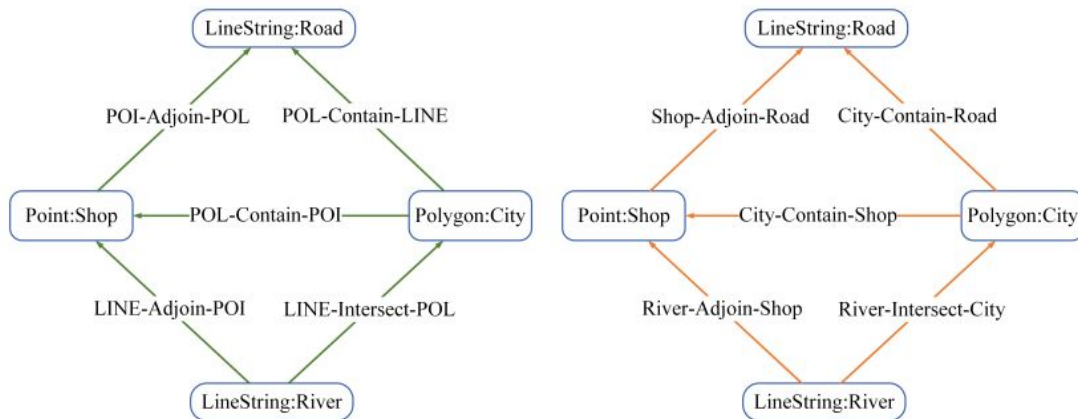
Construction of Geospatial Knowledge Graphs: Modeling Spatial Data (1)

- Geometries as **node attributes** or dedicated **geometry nodes**
 - Geospatial features (points, lines, polygons) become entities (e.g., "Building A", "Road 123") in the knowledge graph.
 - Their spatial footprint (coordinates, bounding boxes, shapes) are encoded either as attributes of those entity nodes or as separate geometry-nodes linked to the feature entity.



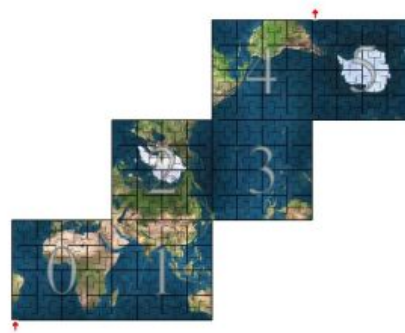
Construction of Geospatial Knowledge Graphs: Modeling Spatial Data (2)

- Spatial relations and topological relationships as edges in the graph
 - Beyond just “has-geometry”, spatial knowledge graphs **can explicitly encode spatial relations between entities**: e.g., “adjacent to”, “contains”, “intersects”, “within”.
 - The graph edges can express both **semantic relations** (e.g., “building A is part of campus X”) and **spatial relations** (e.g., “building A is within polygon campus-X”, “road R intersects road S”).



Construction of Geospatial Knowledge Graphs: Modeling Spatial Data (3)

- Hierarchical and multi-scale dimensions in spatial modelling
 - Many GeoKGs incorporate hierarchical spatial contexts (e.g., country → state → county → city) so that entities can be placed in **nested spatial tiers**.
 - Use of discrete global grid systems (DGGs) like H3 or S2 is emerging for efficient representation, integration and querying of multi-scale spatial data within knowledge graphs. For example, DGG cells can be used as nodes or linking structures in GeoKGs.

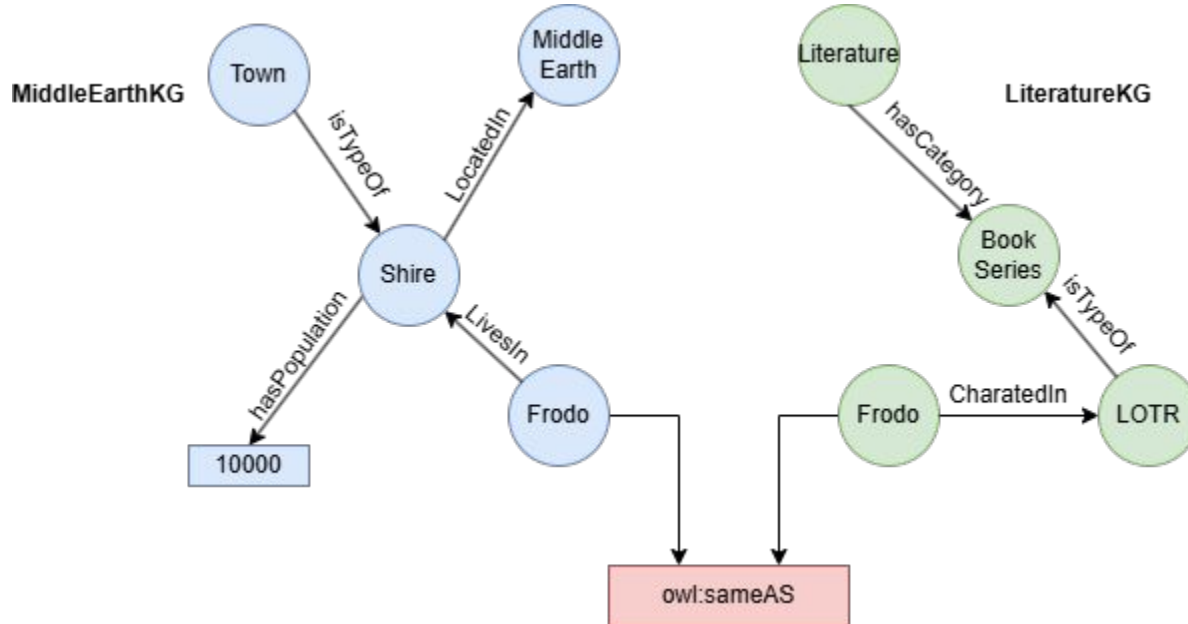


Interlinking in Geospatial Knowledge Graphs (1)

- Keep in mind that KG and GeoKG are not static objects unrelated to one another.
- KGs and GeoKGs represent internet resources that very often interact with each other. (Linked Data!)
- In the same principle, KGs and GeoKGs can “interact” with each other.
- This is where interlinking comes to play!

Interlinking in Geospatial Knowledge Graphs (2)

- Knowledge graph interlinking is the process of connecting entities, concepts, or data points across different knowledge graphs (KGs) or data sources so that they can be integrated, queried, and reasoned about collectively.

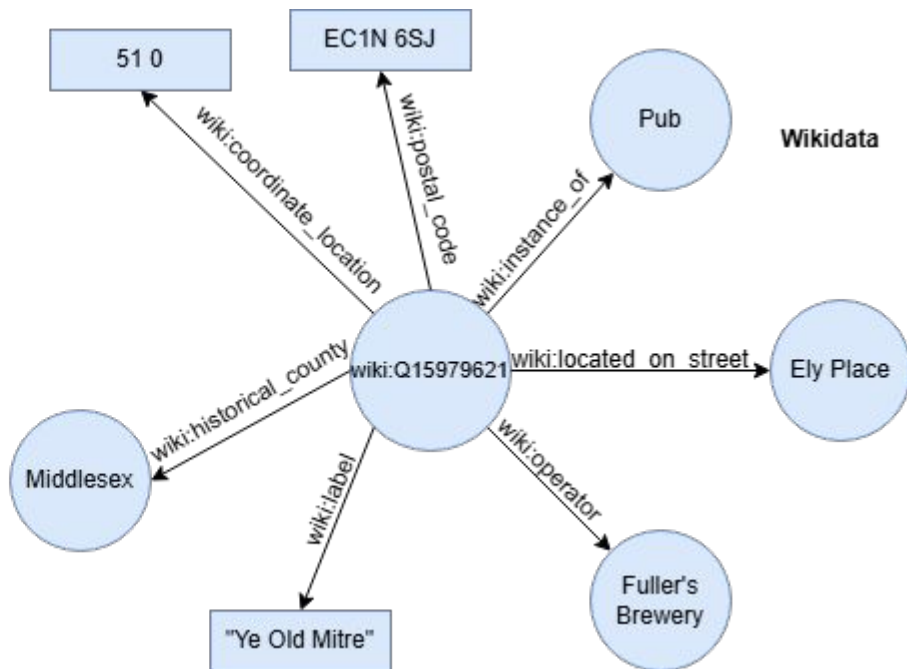
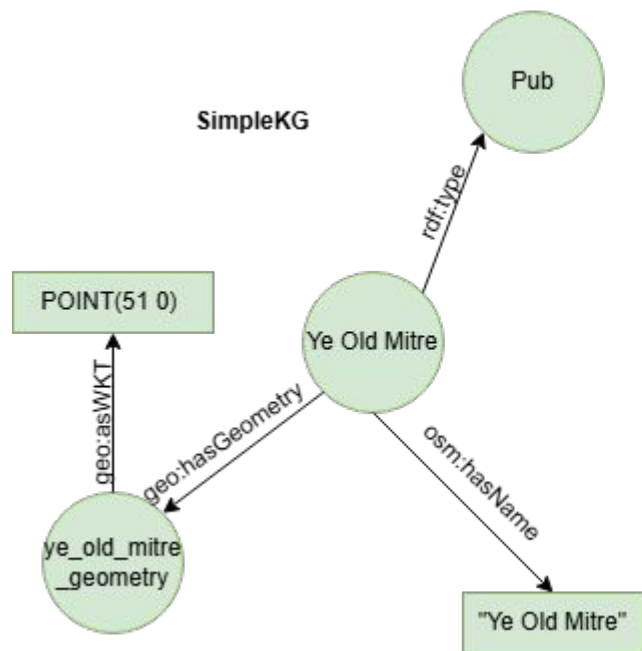


Interlinking in Geospatial Knowledge Graphs (3)

- But why is interlinking useful when using GeoKGs?
 - Because with interlinking we can **combine knowledge!**
- Utilizing interlinking techniques allows us to enrich our geospatial kgs with **external spatial knowledge** or even **thematic knowledge!**
- For example let's say that we have a constructed a GeoKG using only **points-of-interest from OSM**. We have a plethora of entities and their geospatial information
 - However, this is about all we have!
- What if we wanted to enrich this special information with thematic knowledge like, reviews, descriptions, historical references, famous people who visited the POI and many more?

Interlinking in Geospatial Knowledge Graphs (4)

- The best way to do that would be to interlink our GeoKG with another KG that holds our missing thematic knowledge, for example Wikidata!



Interlinking in Geospatial Knowledge Graphs (5)

- By linking our graph with Wikidata we can provide ours nodes with a wide array of information!
- But how can we achieve this interlinking?
- By utilizing interlinking frameworks like LIMES or Silk!



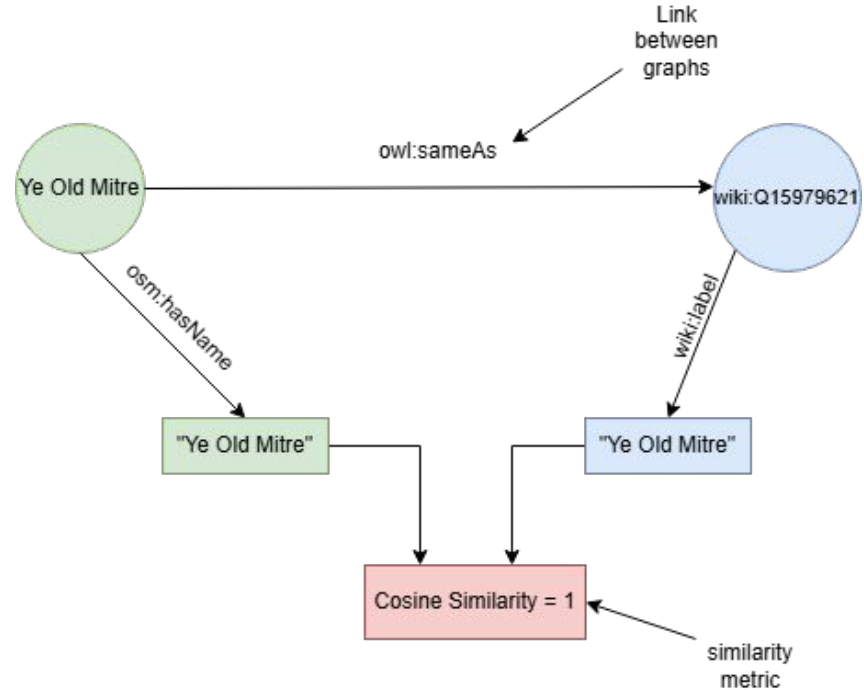
- **LIMES** - A Framework for Link Discovery on the Semantic Web Axel-Cyrille Ngonga Ngomo, Mohamed Ahmed Sherif, Kleanthi Georgala, Mofeed Mohamed Hassan, Kevin Dreßler, Klaus Lyko, Daniel Obraczka, Tommaso Soru
- **Silk**—A Link Discovery Framework for the Web of Data, Julius Volz, Christian Bizer, Martin Gaedke, Georgi Kobilarov

Interlinking in Geospatial Knowledge Graphs (6)

- **LIMES** is a link discovery framework for the Web of Data. It implements time-efficient approaches for large-scale link discovery based on the characteristics of metric spaces. LIMES can be downloaded as standalone tool for carrying out link discovery or as a Java library.
- **Silk** is an open source framework for integrating heterogeneous data sources. The primary use cases of Silk include:
 - Generating links between related data items within different Linked Data sources.
 - Linked Data publishers can use Silk to set RDF links from their data sources to other data sources on the Web.
 - Applying data transformations to structured data sources.

Interlinking in Geospatial Knowledge Graphs (7)

- How do these frameworks work?
 - By utilizing similarity techniques
 - The user can select the **properties between knowledge graphs** and keep the most similar instances of these properties
 - For example in our KG we can see that the predicate **osm:hasName** and Wikidata's **wiki:label** are very similar. We can try to **match entities** based on this two properties!



Interlinking in Geospatial Knowledge Graphs (8)

- These frameworks provide a variety of similarity metrics (Cosine similarity, Levenshtein distance , euclidean distance etc)
- We can pick and choose whatever metric we prefer.
- We can also combine metrics in these frameworks with boolean operations. For example in LIMES syntax:

```
<METRIC>  
  ADD(  
    0.5*levenshtein(x.label, y.label),  
    0.5*Geo_Min(x.polygon, y.polygon, 1000)  
  )  
</METRIC>
```

Interlinking in Geospatial Knowledge Graphs (9)

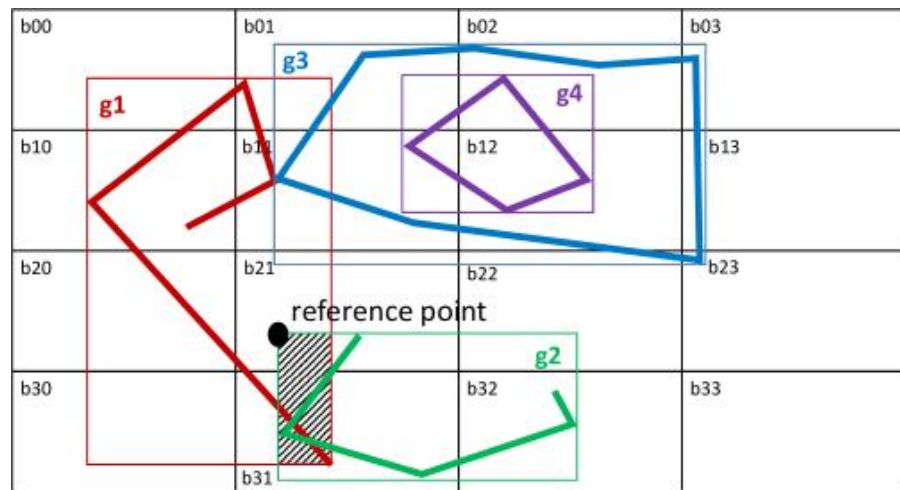
- These frameworks are very useful when working with geospatial graphs, because they also implement metrics that cover spatial operations. Some examples from **LIMES** bellow:
 - **Geo_Hausdorff**: The Hausdorff distance is a measure of the maximum of the minimum distances between pairwise points in the two input geometries.
 - **Geo_Max**: The idea behind this measure is to compute the overall maximal distance between pairwise points of the two input geometries.
 - **Geo_Min**: The idea of Geo_Min is akin to that of Geo_Max and is defined as minimal distance between pairwise points of the two input geometries.
- By using these metrics we can also add spatial parameters when trying to link entities between graphs (in our example we could use the **WKT from osm** and **coordinates_location from wikidata**)

Interlinking in Geospatial Knowledge Graphs (10)

- A thing to keep in mind is that we can utilize a framework like Silk, or more generalized entity resolution frameworks like pyJedAI, to perform interlinking between knowledge graphs and other data sources like CSV, JSON etc
- This process is either supported internally like in Silk, or requires a pre-processing step (like in pyJedAI) where the KG data is transformed into another form like CSV, before the matching begins

Geospatial Interlinking (1)

- What is geospatial interlinking?
 - The discovery of spatial relationships between two sources
- Why does it matter in GeoKGs?
 - If you work with spatial data in general, a fact will very soon become clear. Spatial operations are costly!
 - With geospatial interlinking, we can **precalculate expensive operations** like intersections, containments, etc, to make our applications much faster!



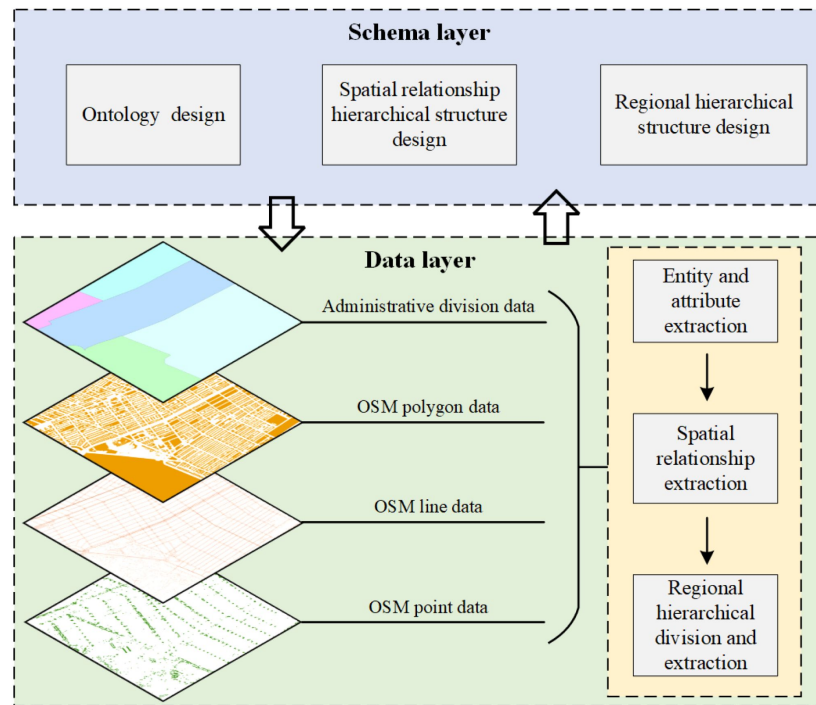
Geospatial Interlinking (2)

- As an example of why these **precalculations** are **vital to GIS applications** we present results from our GeoQuestion1089 paper
- In these table Question Categories are shown as described in the paper
- It can be clearly shown that execution time without **precalculations** (without materialized relationships) degrades when questions get more expensive (like in category D)

Question Category	Number of questions	Average time	Average time materialized
B	12	< 1	< 0.1
C	10	378	< 1
D	13	27237	< 2
E	9	262	< 2
F	8	179	< 1
G	3	100	< 2
H	3	68	< 2
I	2	245	< 2

Geospatial Interlinking (3)

- Geospatial interlinking can also be used to create hierarchical models for the representation between spatial entities
- Through this we can implement the “Spatial relations and topological relationships as edges in the graph” as described previously!



Geospatial Interlinking: JedAI-spatial (1)

- We can achieve geospatial interlinking using a dedicated tool developed by our team: **JedAI-spatial**
- **JedAI-spatial** is an open-source system for computing topological relations according to the DE9IM model between datasets with geometric entities.
- Numerous algorithms aim to improve the time efficiency and scalability of Geospatial Interlinking, often by operating in main memory and limiting comparisons to geometries likely to be related.

Three-dimensional Geospatial Interlinking with JedAI-spatial, Marios Papamichalopoulos, George Papadakis, George Mandilaras, Maria Despoina Siampou, Nikos Mamoulis, Manolis Koubarakis

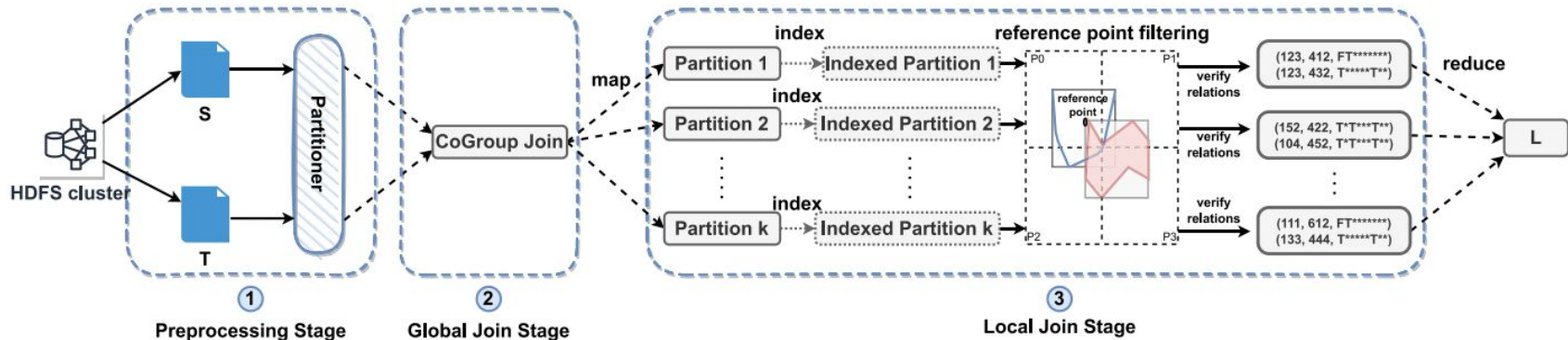
Geospatial Interlinking: JedAI-spatial (2)

- There is currently no open-source framework that unifies these algorithms to support researchers and practitioners in populating the Linked Data cloud with topological relations.
- Existing systems like Silk and LIMES include only their creators' methods, while potentially useful libraries such as stLD are not publicly available.

Geospatial Interlinking: JedAI-spatial (3)

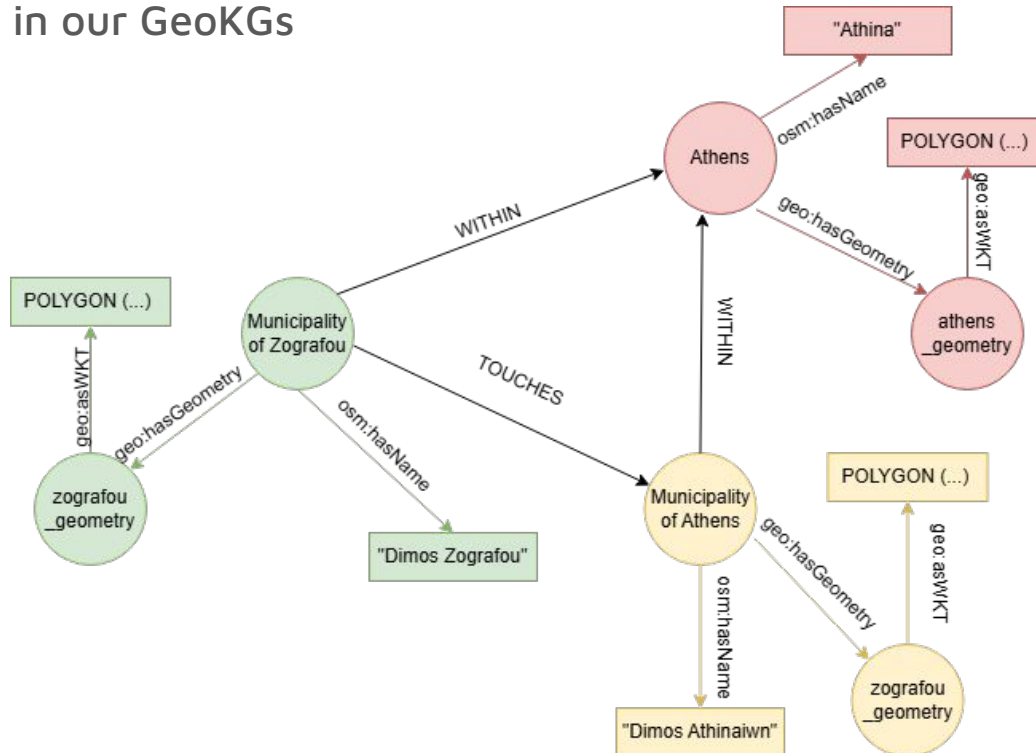
- JedAI-spatial addresses these gaps by providing an open-source platform that organizes, optimizes, and extends state-of-the-art Geospatial Interlinking methods within a unified, modular framework.
- The system offers an intuitive user interface, supports easy integration of new algorithms, and is publicly available at:

<https://github.com/AI-team-UoA/JedAI-spatial>



Geospatial Interlinking: JedAI-spatial (4)

- Using JedAI-spatial we can efficiently calculate and materialize geospatial relationships in our GeoKGs



Geospatial Interlinking: JedAI-spatial (5)

- JedAI-spatial is open source software made to run either serially (Java) or in-parallel (Scala)
- JedAI-spatial functions both with knowledge graph data (NTriples) and semi-structured data sources like CSV, TSV, allowing for diverse geospatial customizations.
- It can be run through a CLI interface or the dedicated GUI.

Geospatial Interlinking: JedAI-spatial (6)

Geometry Loader

Load datasets into Geometries. Only the source dataset will be stored in the memory while the target dataset will be streamed from the Disk.

Source: CSV [Configure](#) [Explore](#)

Target: CSV [Configure](#) [Explore](#)

Separator: \t
ID index: 1
Geometry index: 0

Source: CSV
File: LINEARWATER_UK.csv
Attributes in first row: false

Algorithm Selection

Choose the geospatial interlinking algorithm. In case of Budget Agnostic algorithms, users must provide a budget.

Select Algorithm Type: Budget Agnostic

Select Algorithm: GIA.int

[Previous](#) [Next](#)

Geospatial Interlinking: JedAI-spatial (7)

Geospatial Interlinking Execution

Press "Execute Interlinking" to run the selected algorithm.
You can export the results to a file with the "Export" button.

ResultsDetailsWorkbench

Source Geometries:	1000	Target Geometries:	100000
Total Relations:	5118	Qualifying Pairs:	2106
Verifications:	6040	Total execution time (ms):	2862

Contains:	847	Covers:	906	Covered By:	0
Crosses:	83	Intersects:	2106	Overlaps:	0
Touches:	1176	Equals:	0	Within:	0

Completed

Execute Interlinking

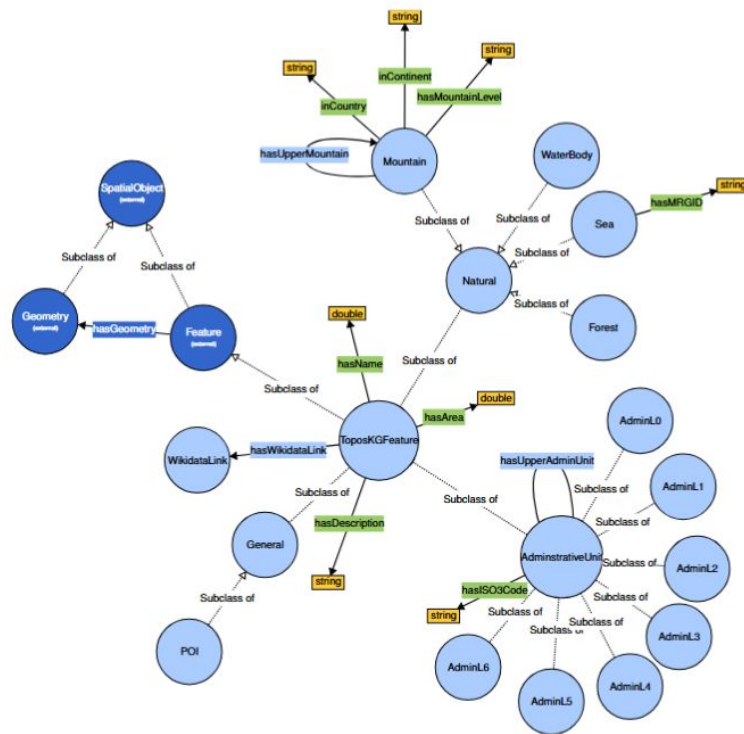
Explore

Show Plot

Start Over

Using the Topos framework to construct GeoKGs (1)

- We will now present the Topos framework, a toolchain for constructing geospatial knowledge graphs based on the ToposKG ontology.
- Through the Topos framework, a user can create and customize a GeoKG



Using the Topos framework to construct GeoKGs (2)

- Using the modular download interface, we can download app-ready knowledge graph data from various geospatial sources (OSM, GAUL, GADM, etc)
- This data can serve as the spatial baseline of our GeoKG

Modular Download

If you want to manually download specific data sources you can do so through this interface:

Data Sources Tree

Download Selected

- ▾ Isle Of Man
- ☐ Israel
- ☐ Italy
- ☐ Jamaica
- ▾ ☐ Japan
 - ☐ Japan_0.nt
 - ☒ Japan_1.nt
 - ☐ Japan_2.nt
 - ☐ Japan_all.nt
- ☐ Jordan
- ☐ Kazakhstan
- ☐ Kenya
- ☐ Kiribati
- ☐ Kosovo

Using the Topos framework to construct GeoKGs (3)

- Through the Topos framework we can parse additional data sources from semi-structured sources (CSV,JSON,GeoJSON, Shapefiles, etc)
- Topos provides parsers and RML mapping processors (through Morph-KGc) to transform input data source into knowledge graph data

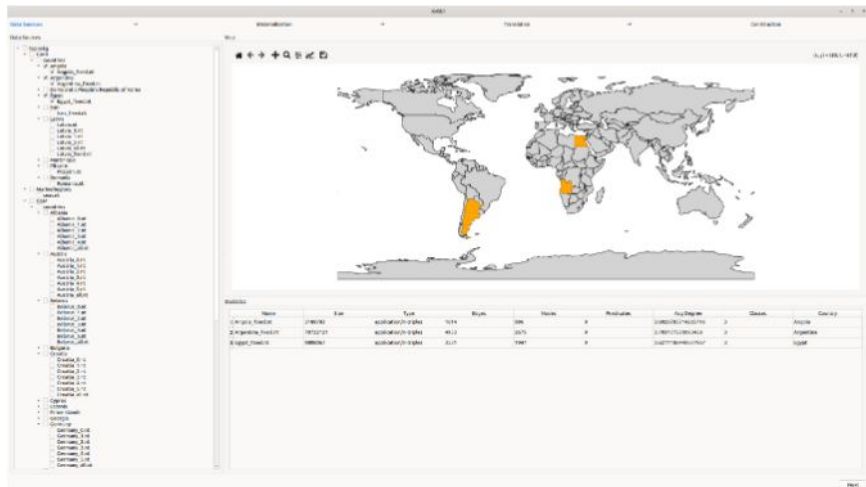
```
#Create the converter for GeoJSON files
converter = GeoJSONConverter(geojson_file,output_nt_file,ontology_uri,resource_uri)
#Parse the input file
converter.parse()
#Export to the output file
converter.export()
```

Using the Topos framework to construct GeoKGs (4)

- Through our parsing library `toposkg-lib`, we can also use the materialization provided by `JedAI-spatial` through the `blueprint` class
- Utilizing all the above components, we provide users with a geospatial knowledge graph construction pipeline with rich spatial information, the ability to add and interlink new thematic and spatial data and to calculate spatial relationships between graph entities

Using the Topos framework to construct GeoKGs (5)

- For more novice users, Topos framework provides a desktop application and a LLM-powered chatbot



LLM-powered chatbot loop

The LLM (currently only available through the OpenAI API) uses function calling to utilize the toposkg library.

A conversation loop is implemented.

Unhide the output to see the conversation and intermediate outputs.

```
[ ] from toposkg.toposkg_lib_fc import *

fc = FunctionCallingController("YOUR_OPEN_AI_API_KEY", "https://toposkg.di.uoa.gr")

while True:
    user_input = input("User: ")
    if user_input.lower() == "exit":
        break
    fc.respond_to_chat(user_input, print_output=True) # Disable print_output to see fewer messages.

Do you want to proceed with downloading the entire knowledge graph sources (100gb)? Any previously downloaded sources will not be redownloaded. (y/n)
Skipping download of sources...
Loading source information from ~/.toposkg/sources_cache
User: What data sources about Greece are available?
Response: function_call(arguments={}, call_id='call_2kGzIjuPbsyV8988xPduV2Vo', name='source_manager_get_source_paths', type='function_call', id='fc-...')
Assistant: There are multiple data sources available about Greece. These include:

1. GAUL country datasets:
- /root/.toposkg/sources_cache/toposkg/GAUL/countries/Greece/Greece_0.nt
- /root/.toposkg/sources_cache/toposkg/GAUL/countries/Greece/Greece_1.nt
- /root/.toposkg/sources_cache/toposkg/GAUL/countries/Greece/Greece_2.nt
- /root/.toposkg/sources_cache/toposkg/GAUL/countries/Greece/Greece_all.nt
```

Thank you!

kpl@di.uoa.gr

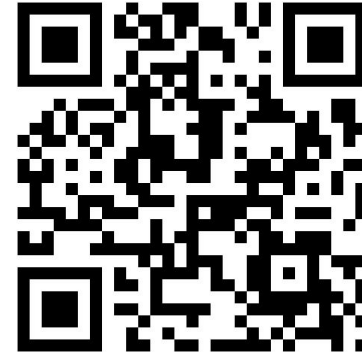
Web Pages



Our group's LinkedIn



Our group's website



Our group's Twitter